

PA3

PA3

- **Goal: Design and implement a reservation server that supports many clients**
 - Server handles the reservation procedure for each client.
 - Client allows users to manage seats by sending a query, containing the user's desired action, to the server.
 - » Important Note: Terms “user” and “client” are different.
- **Deadline: 2026-06-07 (Sun) 23:59**

Restrictions

■ Restrictions

- Project will be compiled and executed in a Linux environment.
- Sockets and IP address are configured as in the lab.
- Resources allocated dynamically should be freed before the program terminates.
 - » Includes files, memory, threads, locks, and child processes.
 - » Can check this using `valgrind --leak-check=full --track-origins=yes --show-leak-kinds=all ./<program name> <args>`
- This is a personal assignment.
 - » You can discuss the task together, but you must write the code yourself.

Restrictions

▪ Submission details

- Latest submission is used for scoring.
 - » Submission time is when this submission was submitted.
- The total score is 100 points, and 10 points are deducted per day (max 4 days).
- Submit two files
 - » A tarball called “[student_id]_pa3.tar.zst” that contains a “pa3” directory, which contains the Makefile and source code.
 - » Report containing the design and implementation of your project formatted as a PDF with the name “student_id_pa3.pdf”.
 - » This report can be written in English or Korean.

Restrictions

■ Submission details

- When compiled, it should generate a server program called “pa3_server” and a client called “pa3_client”.
- The following code contains the expected output when extracting the assignment.
 - » If the following code does not work, your project might not be graded correctly.

```
$ tar xaf 20XXXXXXXXX_pa3.tar.zst
$ cd pa3
$ make
$ ls | grep pa3
pa3_server pa3_client (other files_with `pa3`)
```

Number of Cores

- **Starting this week, we will be working with concurrent applications**
 - Number of cores should be increased as much as possible (i.e., somewhere within the green, yellow range in VirtualBox). At least 4 cores is recommended.
 - The number of cores is referred to as NUM_CORES.

Number of Available Cores

- Getting number of cores in C:
 - » Based on <https://stackoverflow.com/a/74744791>, which contains multiple approaches as well.
 - » For this approach, we get the set of cores the application can run on using `sched\_getaffinity`, and we store it in `cpu_set`.
 - » Needs `_GNU_SOURCE` to be defined (either in Makefile or C code)
 - » `sched_getaffinity` accepts pid, `cpu_set` size, and the set/mask.
 - » Then, we use `CPU_COUNT_S` to count the number of cores in `cpu_set`.

```
#ifndef _GNU_SOURCE
    #define _GNU_SOURCE
#endif

#include <sched.h>
int get_num_cores() {
    cpu_set_t cpu_set;
    sched_getaffinity(0, sizeof(cpu_set), &cpu_set);
    return CPU_COUNT_S(sizeof(cpu_set), &cpu_set);
}
```

Prerequisites

- **Install the following packages**

- `sudo apt install libargon2-dev`

- **In the Makefile**

- add “-D_GNU_SOURCE” in the gcc options
- add “-largon2” at the end of your linking command
 - » (i.e. ``gcc file.c -largon2`` or ``gcc file.o -largon2``)

User

- Identified with a username string.
- There can be a maximum of 10,000 users.
- A user can only use one client.
 - » If User 1 is currently logged in through a client, other attempts to log in as User 1 from another client will be rejected.
- A client can only accommodate one user.
 - » The client can only send queries of user 1 until user 1 successfully logs out.
- A user associated with a client is called an active user.
- A client associated with a user is an active client.

Server

- The server manages 100 seats in total (range: 1~100).
- Each seat should be managed by the server using at least one synchronization mechanism and one synchronization variable.
- To lessen the burden on your system, the server uses NUM_CORES of threads.
 - » However, the number of active users can be larger than the number of threads.
 - » Each thread will poll its own set of clients.
 - » When a new client connects, its associated socket will be polled by the thread with the fewest number of clients.
 - » This means when 4 users connect to a server with 4 cores, there should be 4 threads competing for the seats, requiring synchronization.

Server

- Server should check if the user is logged in when the user attempts to book a seat.
- Server should send a response, containing the response code and the data, to the client after it receives a query from the client.
- If the server receives an action not known to the server, it should print “Invalid action received: ACTION”, where ACTION is the action given by the user.
- Server output does not matter.
 - » You can leave debugging code if needed.
- Running server: `./pa3_server port`
- Skeleton code is provided
 - » Need to modify both `handle_request.c` and `pa3_server.c`

Client

■ Two ways to interact with client

- **File mode:** file is passed as a command-line argument
 - » This file contains one or more requests/queries
 - » One line = request
 - » Format: ACTION_NAME [arguments]
 - » When all the requests in the file are processed, the client terminates.
- **Interactive mode: REPL**
 - » Client runs on infinite loop until the user enters “exit”.
- Running client: `./pa3_client ip_address port [file]`
- Skeleton code is provided
 - » Just need to modify `pa3_client.c`

Data Structures

- You can use the following structures for sending data from the server and client.
 - Request is sent by the client and received by the server.
 - Response is sent by the server and received by the client.

```
struct Request {  
    uint64_t username_length;  
    uint64_t data_size;  
    Action action;  
    char* username;  
    char* data;  
};
```

```
struct Response {  
    uint64_t data_size;  
    int32_t code;  
    uint8_t* data;  
};
```

Sending Data

- **To send data with unknown length, we use TLV (type-length-value) or LTV (length-type-value)**
- **Safest way to do this is**
 - Client sends type, length(s), and value(s) individually.
 - Server receives the type and the length individually.
 - Server can then receive the value(s) as the length is already known.
 - Use `safe_write/safe_read` instead of `write/read`
- **Type of response data is a sequence of bytes (`uint8_t[]`), which can be safely casted using `memcpy`.**

Synchronization

- Each thread handle their own set of clients.
- Clients may request the same seat at the same time.
 - To prevent races, seat operations (reading/modifying seat fields) must be protected by a mutex.
- Same goes for `User` and `Users` structures.
- To avoid deadlock, always lock in the same order (e.g., user-array lock -> user lock -> seat lock).
- Mutex for poll set should not be held during socket read/write.

Actions

Actions (Client)

- There are seven types of actions.

Action ID	Enum Name	Description	Data field
0	TERMINATION	Terminates the connection between the client and the server	N/A
1	LOGIN	Attempt to log in	Password
2	BOOK	Attempt to book a seat	Seat number
3	CONFIRM_BOOKING	Check the number of seats booked by the user	“available” or “booked”
4	CANCEL_BOOKING	Cancel the user’s booking if the user has booked a seat	Seat number
5	LOGOUT	Log out	N/A (no data passed)
6	QUERY	Get details regarding a seat	Seat number

Actions (Server)

- **Server returns a response code, which is an enum value referring to the error (0 on success).**
- **If action id is not in the range of [1, 6], return -1.**
 - Server should not receive termination (0) action.

Action ID	Name	Data value
1	Log in	N/A
2	Book	The booked seat number (e.g. 3)
3	Confirm booking	List of available/booked seat numbers (e.g. 3,4,5,6)
4	Cancel booking	The canceled seat number (e.g. 3)
5	Log out	N/A
6	Query	Query struct

Actions (Print format)

- On the client, print to stdout on success and to stderr on failure.
- ERROR_MSG can be found in the Errors section.

Action ID	Name	On success	On failure
0	Termination	Connection terminated.	[Refer to slide]
1	Log in	User USER logged in successfully.	[Refer to slide]
2	Book	Seat SEAT_NUMBER was booked successfully by user USER!	[Refer to slide]
3	Confirm booking	[Refer to slide]	[Refer to slide]
4	Cancel booking	Booking for seat SEAT_NUMBER was canceled successfully by user USER!	[Refer to slide]
5	Log out	User USER logged out successfully.	[Refer to slide]
6	Query	Seat SEAT_NUMBER was booked n times and cancelled n times!	[Refer to slide]
<Other>	Unknown action	-	Invalid action received!

Action 0: Termination

- **Termination: Terminates connection and client**
 - Despite being an action, the client does not send a termination request to the server.
 - » If the server somehow receives this request (i.e., wrong implementation), the server should return -1 and the client should print “Invalid action received: 0”.
 - If a user is currently logged in, send a logout request and then print the output of the logout response in the format described in the logout slide.
 - The client closes the connected socket and the server must handle the closed connection accordingly (remove it from poll set, etc.).

Action 0: Termination

- **Termination: Terminates connection and client**
 - In **file mode**, the client automatically terminates once all actions have been performed.
 - In **interactive mode**, the user must enter “exit” to terminate the client.

Action 1: Login

- **Login: assign user as client's active user**
 - Query format: "login username password"
 - **Request Data:** variable-length password.
 - Response Data: N/A.
 - A user who has executed this action successfully is referred to as an active user.
 - A client associated with an active user is referred to as an active client.
 - Action makes the requested user the client's active user if successful.

Action 1: Login

▪ Handling users

- Users and their associated passwords will be stored in memory (restarting the server wipes user data).
- User has the following struct and is stored in an array.

```
typedef struct {  
    const char* username;  
    const char* hashed_password;  
    bool logged_in;  
} User;
```

- If user is not in the array, register the user by adding the user to the array along with their hashed password and set the logged_in to true.
- If user is in the array, compare the passwords and allow the user to log in if they are identical.

Action 1: Login

▪ Handling passwords

- Storing passwords in plaintext is a bad habit, so the password will be hashed before storing it in the array.
- Two functions are provided, but **argon2** has to be installed and linked (see prerequisites).
 - » “hash_password”: saves a 128-byte hashed password to the array passed as an argument
 - » “validate_password”: checks if the passed plain-text password matches the hashed password.

Action 1: Login

- **If the user is new,**

- Call “hash_password” in the skeleton code.

- » `hash_password(char* password, char* hashed_password)`

- » password: Where your plaintext password is located

- » hashed_password: Where your 128-byte hashed password will be saved.

- Add the user with the username and hashed password to the users array.

Action 1: Login

- **If the user exists,**

- Find the user id (i.e., index) in the users array and get the user object.
- Compare the two passwords using the function “validate_password”, which can be found in the skeleton code.

```
» validate_password(char* password_to_validate, char* hashed_password)
```

- » password_to_validate: Where the password passed by the user is located.
- » hashed_password: Where your hashed password is located.
- Returns a Boolean value (true if the password is correct, and false otherwise).

Action 1: Login

▪ Errors

Response code	Error message	Description
SUCCESS	User USER logged in successfully!	user logged in successfully
ACTIVE USER	User USER is already logged in!	an active user attempts to log in when already logged in on another client
ACTIVE CLIENT	Client is already serving user USER!	any user, including the active user, tries to log in on an active client
INCORRECT PASSWORD	User USER provided incorrect password!	a registered user attempts to log in with the wrong password
NO PASSWORD	User USER did not provide a password!	there is no data in the data field

Action 2: Book

- **Book: Books seat for the user**
 - Query format: “book seatnumber”
 - Seat number has to be between 1 and 100.
 - » (inclusive, i.e., [1, 100]).
 - Request data: booked seat number
 - Response data: N/A

Action 2: Book

▪ Errors

- Note: * - If USER is not known, it is not printed.

Response code	ERROR_MSG	description
SUCCESS	Seat SEAT_NUMBER was booked successfully by user USER!	user booked seat successfully
USER NOT LOGGED IN	User USER* is not logged in!	user attempts to book when it is not the active user in the client
SEAT UNAVAILABLE	Seat SEAT_NUMBER is unavailable for booking!	seat requested by the active user is already booked by another user (including the user)
SEAT OUT OF RANGE	Seat SEAT_NUMBER is out of range [1,100]!	seat number is out of the range (includes errors due to non-numeric data).
NO DATA	Please enter a seat number to book!	there is no data in the data field

Action 3: Confirm booking

- **Confirm booking: Check available/booked seats**
 - Query format: “confirm-booking available/booked”
 - Request data field: “available” or “booked”
 - » If request’s data field is “available”, server returns all available seats.
 - » If request’s data field is “booked”, returns seats booked by the user.
 - Response data: array of available/booked seats
 - Client must print the booked seats in ascending order.

Action 3: Confirm booking

▪ Errors

Response code	ERROR_MSG	description
SUCCESS	Available seats: seat0, seat1, seat2	seats the user can book
	Booked seats by user USER: seat0, seat1, seat2	seats the user has booked
USER NOT LOGGED IN	User USER* is not logged in!	user attempts to confirm booking when it is not the active user in the client
INVALID DATA	Invalid data provided for booking confirmation!	user passed data that is not “available” or “booked”.
NO DATA	Please enter 'available' or 'booked' for booking confirmation!	there is no data in the data field

Action 4: Cancel booking

- **Cancel booking: Removes specified seat from user's booked seats.**
 - Query format: cancel-booking seatnumber
 - Seat number has to be between 1 and 100.
 - » (inclusive, i.e., [1, 100]).
 - Request data field: the canceled seat number.

Action 4: Cancel booking

▪ Errors

Response code	ERROR_MSG	description
SUCCESS	Booking for seat SEAT_NUMBER was canceled successfully by user USER!	booking was canceled successfully
USER NOT LOGGED IN	User USER* is not logged in!	user attempts to cancel booking when it is not the active user in the client
SEAT NOT BOOKED BY USER	Seat SEAT_NUMBER was not booked by user USER!	seat was not booked by the user
SEAT OUT OF RANGE	Seat SEAT_NUMBER is out of range [1,100]!	seat number is out of the range (includes errors due to non-numeric data).
NO DATA	Please enter a seat number to cancel booking!	there is no data in the data field

Action 5: Log out

- **Log out: Logs the user out**

- Query format: logout
- Request data: N/A
- Response data: N/A
- Removes the user's status as active user in the client. Also sets the user as inactive in the server.

Action 5: Log out

▪ Errors

Response code	ERROR_MSG	description
SUCCESS	User USER logged out successfully!	booking was canceled successfully
USER NOT FOUND	User USER not found!	user has not registered yet
USER NOT LOGGED IN	User USER* is not logged in!	user attempts to log out when it has not logged in yet

Action 6: Query

- **Query: Prints information regarding a seat.**
 - Query format: “query seatnumber”
 - Request data: seat number
 - Response data: SeatStats struct
 - » Do not convert to string
 - » Size should be the size of the structure.

Action 6: Query

▪ Errors

Response code	ERROR_MSG	description
SUCCESS	Seat SEAT_NUMBER was booked n time(s) and canceled n time(s)!	Seat was queried successfully.
SEAT OUT OF RANGE	Seat SEAT_NUMBER is out of range [1,100]!	seat number is out of the range (includes errors due to non-numeric data).
NO DATA	Please enter a seat number to query!	there is no data in the data field

Examples of client/server

Example

	<Client>	<Terminal>
	[user action data]	
The user has not logged in yet	book 12	User is not logged in!
so these queries will fail.	logout	User is not logged in!
	confirm-booking	User is not logged in!
User user logged in with 'password'	login user password	User user logged in successfully!
User user booked seat 42	book 42	Seat 42 was booked successfully by user user!
Seat 1025 is out of range -> fail	book 1025	Seat 1025 is out of range [1,100]!
10 is an unknown action -> fail	aaa	Invalid action received!
User user logged out	logout	User user logged out successfully!
User AzureDiamond logged in with 'hunter2'	login AzureDiamond hunter2	User AzureDiamond logged in successfully!
Seat 42 already booked -> fail	book 42	Seat 42 is unavailable for booking!
User AzureDiamond booked seat 13	book 13	Seat 13 was booked successfully by user AzureDiamond!
User AzureDiamond booked seat 98	book 98	Seat 98 was booked successfully by user AzureDiamond!
User AzureDiamond confirms the booking	confirm-booking booked	Booked seats by user AzureDiamond: 13, 98
User AzureDiamond logs out	logout	User AzureDiamond logged out successfully!
User user logs in with the wrong password	login user abc	User user provided incorrect password!

Example

<Client 1>	<Client 2>	Description (not error message!)
login user pw		
	login user pw	Failed as user is already logged in on client 1
	login AzureDiamond hunter2	
book 42		
	book 42	Failed as user has already booked seat 42
cancel-booking 42		
	book 42	
	query 42	Should show that seat 42 was booked 2 times and canceled 1 time.
logout	logout	
	login user hunter2	Failed as user's password is pw

Example

- **The input and expected output for the first example can be found in the skeleton code.**
 - To generate the stdout.txt and stderr.txt files, you can run **`./pa3_client IP_ADDRESS PORT pa3_example1.stdin.txt > stdout.txt 2> stderr.txt`**.
- **For the second example, the client script and the expected output are included in the skeleton code.**
 - To run the client script, you must run the server first.